定时任务调度方案

> 文档版本: v1.0
>
> 更新日期: 2026-04-08
>
> 对应架构: ARCH v1.4 `backend/lib/jobs/`

一、技术选型

1.1 方案：Dart 进程内 Cron 调度

使用 `package:cron` （pub.dev）在后端进程内运行定时任务，不引入外部调度器。

理由:

- 单实例部署（Phase 1 仅 1 台服务器），无分布式锁需求
- Dart 原生 async/await，无序列化开销
- 任务执行日志写入 `job_execution_logs` 表，可追溯

依赖:

```
# backend/pubspec.yaml
dependencies:
  cron: ^0.6.1  # 内嵌 cron 表达式调度
```

1.2 备选方案（Phase 2 扩展）

若后续多实例部署，可迁移至 PostgreSQL Advisory Lock + `pg_cron` 或外部 Kubernetes CronJob。

二、调度器架构

2.1 目录结构

```
backend/lib/jobs/
├── job_runner.dart            # 调度器入口，注册所有定时任务
├── job_base.dart              # 任务基类（含通用 try/catch + 日志记录）
├── job_execution_log.dart     # job_execution_logs 表 Repository
├── retry_scheduler.dart       # 失败重试调度器
├── tasks/
│   ├── contract_expiry_check_task.dart   # 合同到期检查
│   ├── invoice_overdue_check_task.dart   # 账单逾期检查
│   ├── billing_generation_task.dart      # 自动账单生成
│   ├── alert_engine_task.dart            # 预警引擎
│   ├── credit_rating_task.dart           # 信用评级重算
│   └── deposit_refund_reminder_task.dart # 押金退还提醒
```

2.2 任务基类

```
/// backend/lib/jobs/job_base.dart
abstract class ScheduledJob {
  /// 任务名称（写入 job_execution_logs.job_name）
  String get jobName;

  /// 执行逻辑，子类实现
  Future<void> execute({String? scope});

  /// 最大重试次数
  int get maxRetries => 3;

  /// 重试间隔（指数退避基数，秒）
  int get retryBaseSeconds => 60;
}
```

2.3 调度器入口

```
/// backend/lib/jobs/job_runner.dart
import 'package:cron/cron.dart';

class JobRunner {
  final Cron _cron = Cron();
  final JobExecutionLogRepository _logRepo;

  JobRunner(this._logRepo);

  void start() {
    // ── 每日 01:00 UTC ──
    _cron.schedule(Schedule.parse('0 1 * * *'), () async {
      await _run(ContractExpiryCheckTask());
      await _run(InvoiceOverdueCheckTask());
      await _run(AlertEngineTask());
      await _run(DepositRefundReminderTask());
    });

    // ── 每月 1 日 02:00 UTC ──
    _cron.schedule(Schedule.parse('0 2 1 * *'), () async {
      await _run(CreditRatingTask());
    });

    // ── 每日 00:30 UTC（账单生成，需在逾期检查之前）──
    _cron.schedule(Schedule.parse('30 0 * * *'), () async {
      await _run(BillingGenerationTask());
    });
  }

  Future<void> _run(ScheduledJob job) async {
    final logId = await _logRepo.create(jobName: job.jobName, status: 'running');
    try {
      await job.execute();
```

```
      await _logRepo.complete(logId, status: 'success');
    } catch (e, st) {
      await _logRepo.complete(logId, status: 'failed', error: '$e\n$st');
      await _scheduleRetry(job, logId, retryCount: 1);
    }
  }

  Future<void> _scheduleRetry(ScheduledJob job, String logId, {required int retryCount}) async
{
    if (retryCount > job.maxRetries) {
      // 超过最大重试次数，等待人工补偿
      await _logRepo.update(logId, status: 'retry_exhausted');
      return;
    }
    final delay = Duration(seconds: job.retryBaseSeconds * retryCount);
    Future.delayed(delay, () async {
      await _logRepo.update(logId, status: 'retry_scheduled', retryCount: retryCount);
      try {
        await job.execute();
        await _logRepo.complete(logId, status: 'success');
      } catch (e) {
        await _scheduleRetry(job, logId, retryCount: retryCount + 1);
      }
    });
  }

  void stop() => _cron.close();
}
```

三、任务清单与调度计划

#	任务名称	Cron 表达式	执行时间 (UTC)	说明
J1	`billing_generation`	`30 0 * * *`	每日 00:30	扫描应生成账单的合同，生成 `draft` 账单
J2	`contract_expiry_check`	`0 1 * * *`	每日 01:00	`active` → `expiring_soon`（≤90 天）；`expiring_soon` → `expired`（已过期）
J3	`invoice_overdue_check`	`0 1 * * *`	每日 01:00	`issued` → `overdue`（已过 due_date）
J4	`alert_engine`	`0 1 * * *`	每日 01:00	触发 8 种预警类型，写入 `alerts` 表 + 发送通知
J5	`deposit_refund_reminder`	`0 1 * * *`	每日 01:00	合同终止前 7 天提醒财务处理押金退还
J6	`credit_rating_recalc`	`0 2 1 * *`	每月 1 日 02:00	重算所有租户信用评级（A/B/C）

执行顺序依赖

```
00:30  billing_generation        ← 必须先于逾期检查
01:00  contract_expiry_check     ← 更新合同状态
01:00  invoice_overdue_check     ← 依赖最新账单数据
01:00  alert_engine              ← 依赖合同+账单最新状态
01:00  deposit_refund_reminder   ← 依赖合同终止日期
02:00  credit_rating_recalc      ← 月初单独执行，无强依赖
```

四、任务逻辑概要

J1 billing_generation（自动账单生成）

```
1. 查询所有 active/expiring_soon 合同
2. 对每个合同：
   a. 计算下一账期（基于 payment_cycle_months）
   b. 如果未生成该账期账单 → 创建 draft 账单
   c. 免租期内 → 创建 exempt 账单
   d. 含租金递增 → 调用 rent_escalation_engine 计算当期租金
3. 批量写入 invoices + invoice_items
```

J2 contract_expiry_check（合同到期检查）

```
1. SELECT * FROM contracts WHERE status = 'active' AND end_date - CURRENT_DATE <= 90
   → UPDATE status = 'expiring_soon'
2. SELECT * FROM contracts WHERE status = 'expiring_soon' AND end_date < CURRENT_DATE
   → UPDATE status = 'expired'
   → 释放关联单元（units.current_status → vacant）
```

J4 alert_engine（预警引擎）

```
对每种 alert_type：
1. 构造查询条件（如 lease_expiry_90: end_date - NOW() BETWEEN 89 AND 91 天）
2. 排除已触发记录（同合同同类型同日不重复，防轰炸）
3. 写入 alerts 表
4. 发送通知（in_app + email）
   - 失败自动重试 ≥3 次
5. 更新 notified_via 数组
```

J6 credit_rating_recalc（信用评级重算）

```
对每个租户：
1. 统计过去 12 个月逾期次数 / 最长单次逾期天数
2. 评级规则：
```

```
  - A: 逾期≤1 次且单次≤3 天
  - B: 逾期2~3 次或单次4~15 天
  - C: 逾期≥4 次或单次>15 天
3. 签约满3 个月后首次评级，新租户默认B
4. UPDATE tenants SET credit_rating, last_rating_date, times_overdue_past_12m,
max_single_overdue_days
```

五、人工补偿机制

5.1 API 端点

端点	方法	权限	说明
`GET /api/jobs/executions`	GET	`system.admin`	查看任务执行日志（可按 status/job_name 过滤）
`POST /api/jobs/executions/:id/retry`	POST	`system.admin`	手动重跑失败任务
`POST /api/jobs/:jobName/trigger`	POST	`system.admin`	手动触发指定任务（如补跑上月信用评级）

5.2 失败重试策略

参数	值	说明
最大自动重试	3 次	超过后标记 `retry_exhausted`
重试间隔	指数退避	60s → 120s → 180s
人工重试	不限次	通过 API 补偿触发

六、启动集成

```dart
/// backend/bin/server.dart
void main() async {
  final config = AppConfig.fromEnvironment();
  final db = await connectDatabase(config.databaseUrl);

  // ... 注册路由、中间件 ...

  // 启动定时任务调度器
  final jobRunner = JobRunner(JobExecutionLogRepository(db));
  jobRunner.start();
```

```
  // 优雅关闭
  ProcessSignal.sigint.watch().listen((_) {
    jobRunner.stop();
    db.close();
    exit(0);
  });
}
```